Recibe una cálida:

# ¡Bienvenida!

Te estábamos esperando 😁

alkemy

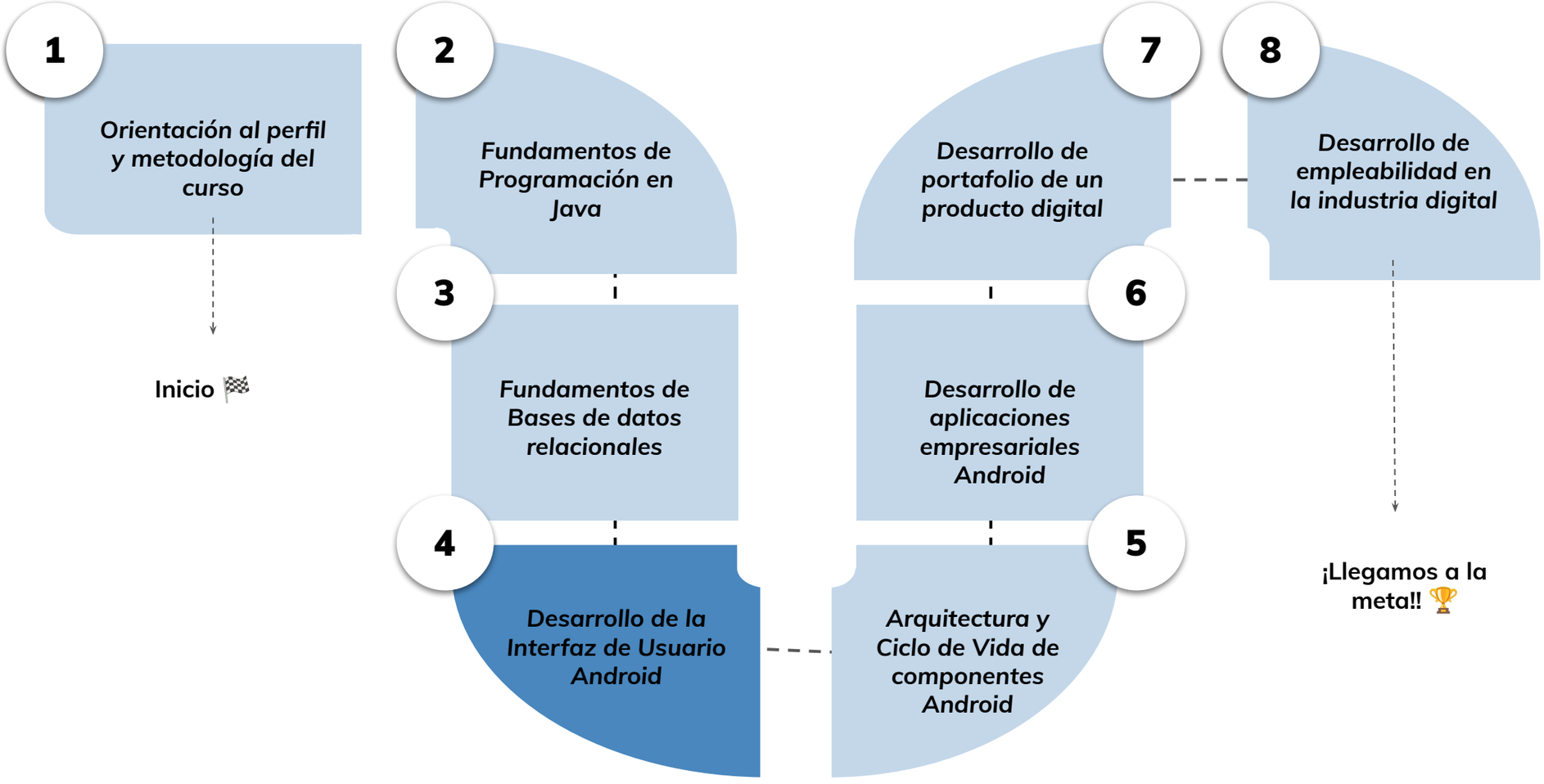
Clase grabada

# Elementos de la interfaz de usuario

**AE5.1:** Utilizar elementos de navegación e interacción de usuario disponibles en el entorno ndroid studio para dar solución a un requerimiento.

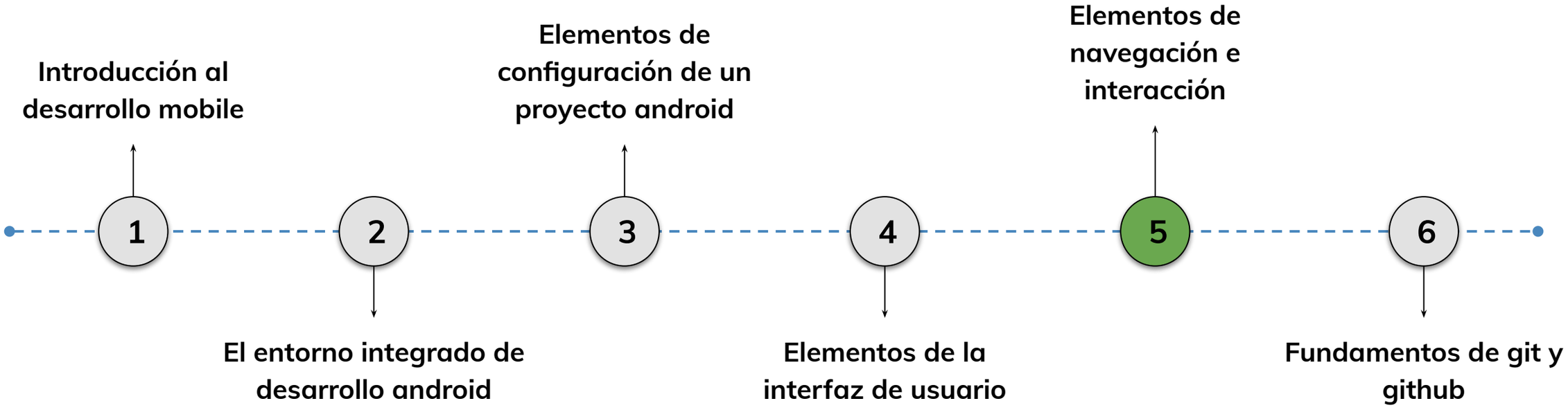
# Hoja de ruta

*¿Cuáles **módulos** conforman el programa?*



# Roadmap de lecciones

*¿Cuáles **lecciones** estaremos estudiando en este módulo?*



# Repaso clase anterior

*¿Quedó alguna duda?*

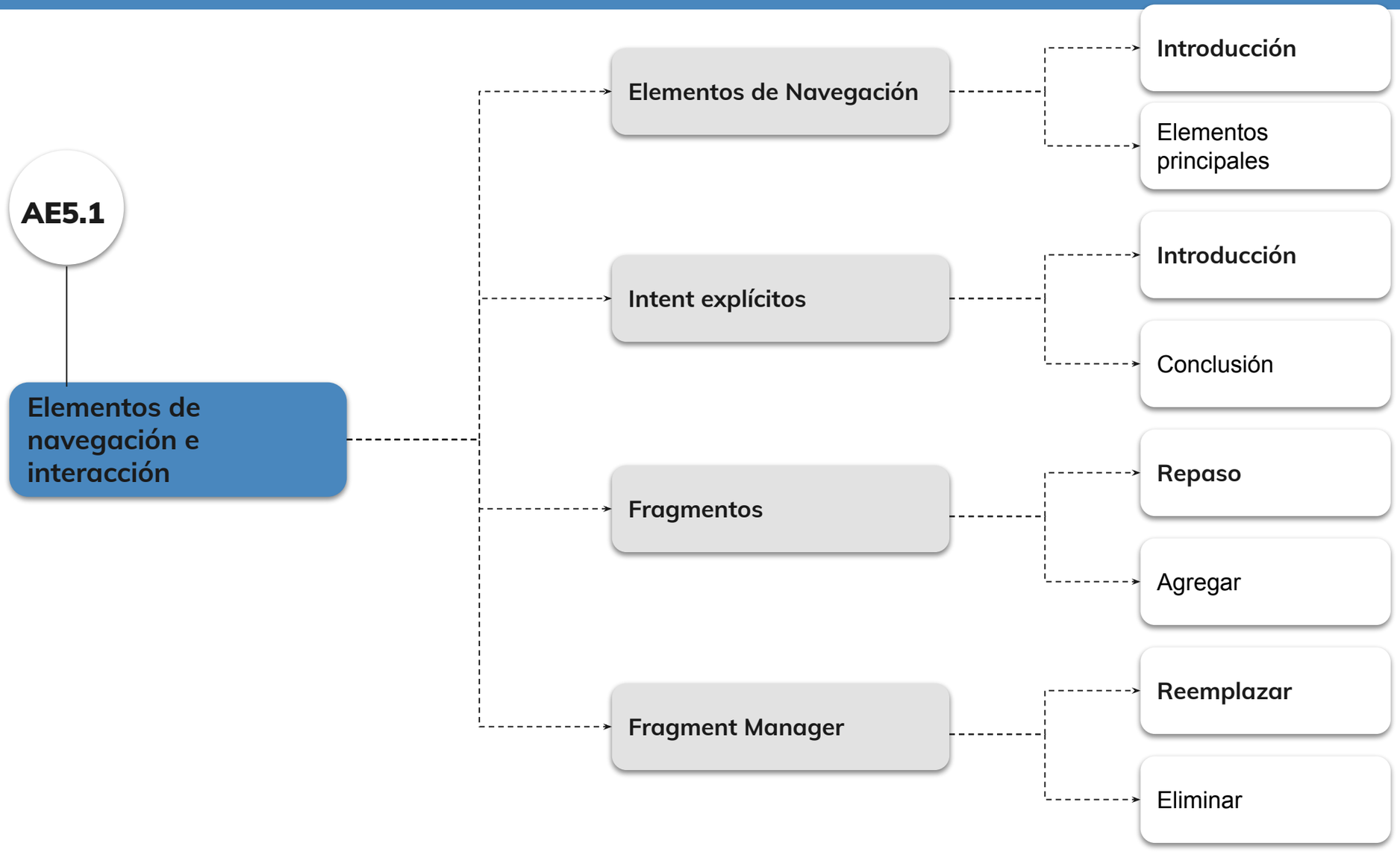
En la clase anterior trabajamos 📚 :

- ✓ *Que es Figma*
- ✓ *Crear vistas usando Firma*
- ✓ *Pasos para la implementación de vistas*

alkemy

# Learning Path

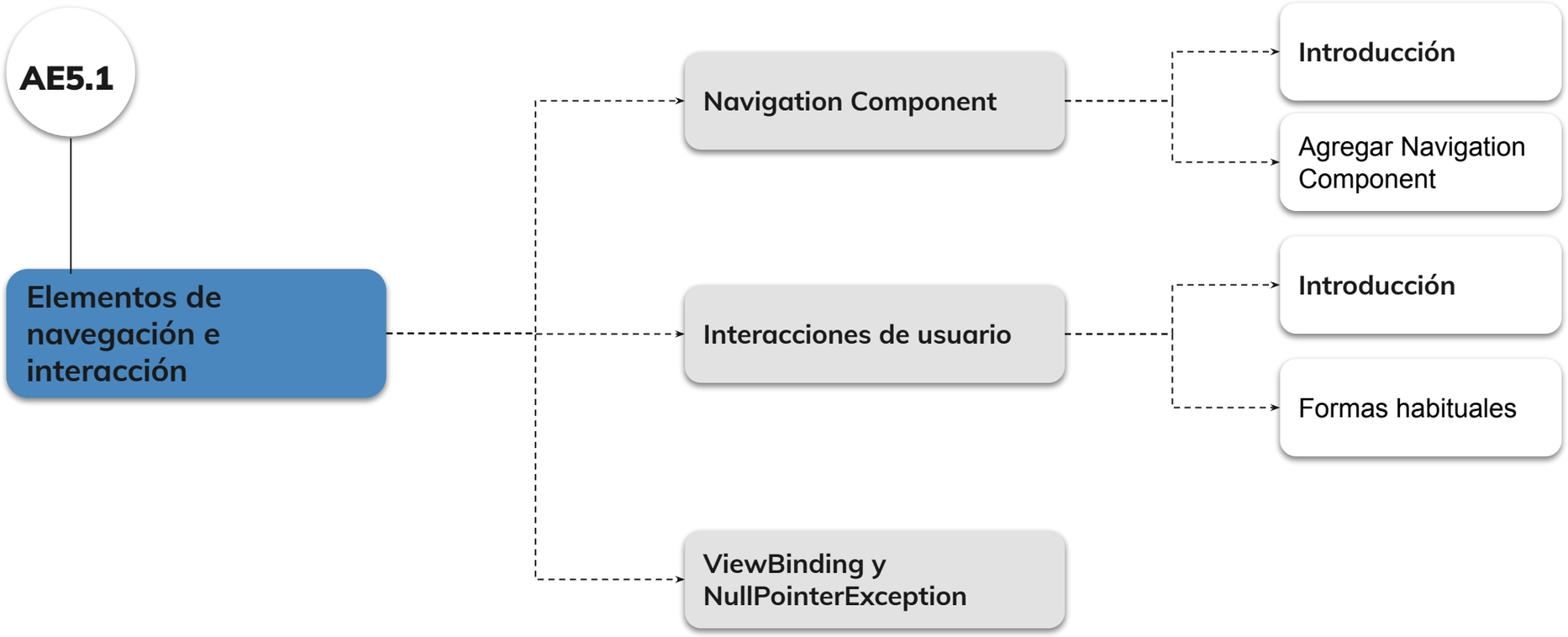
*¿Cuáles temas trabajaremos hoy?*




alkemy

# Learning Path

*¿Cuáles temas trabajaremos hoy?*



# Objetivos de aprendizaje

*¿Qué aprenderás?*

- Entender cuáles son los métodos y elementos en la navegaciones
- Comprender cuáles cómo funcionan los intents Explícitos
- Ver cómo es el comportamiento de la navegación
- Entender como se usar los Framgentos y como se navegan
- Comprender el Fragment Manager y las transacciones
- Entender cómo funcionan los Navigation Components
- Comprender cómo capturar las interacciones de los usuarios
- Cómo trabajamos la relación entre ViewBinding y NullPointerException

# < > Rompehielo🥶

**Contexto:** 🙌

Cuando creamos aplicaciones móviles vamos a querer que el usuario navegue entre varias pantallas y mostrarle diversas cosas para crear el comportamiento que deseamos

**Consigna:** ✍️

¿Cuáles creen que son las diferencias entre por ejemplo un botón que lanza una actividad distinta y un pantalla que tiene pestañas que navega entre diferentes vistas pero no parece cambiar de pantalla?

# Elementos de Navegación

# Elementos de Navegación

Introducción:

Los elementos de navegación en Android, especialmente a partir de la versión 3.0, se han vuelto fundamentales para mejorar la interacción del usuario al permitirles desplazarse de manera más fluida y coherente entre las distintas secciones de la aplicación. Esto se logra gracias a la inclusión del componente Navigator, que ofrece una solución flexible y coherente para gestionar la navegación en la aplicación.

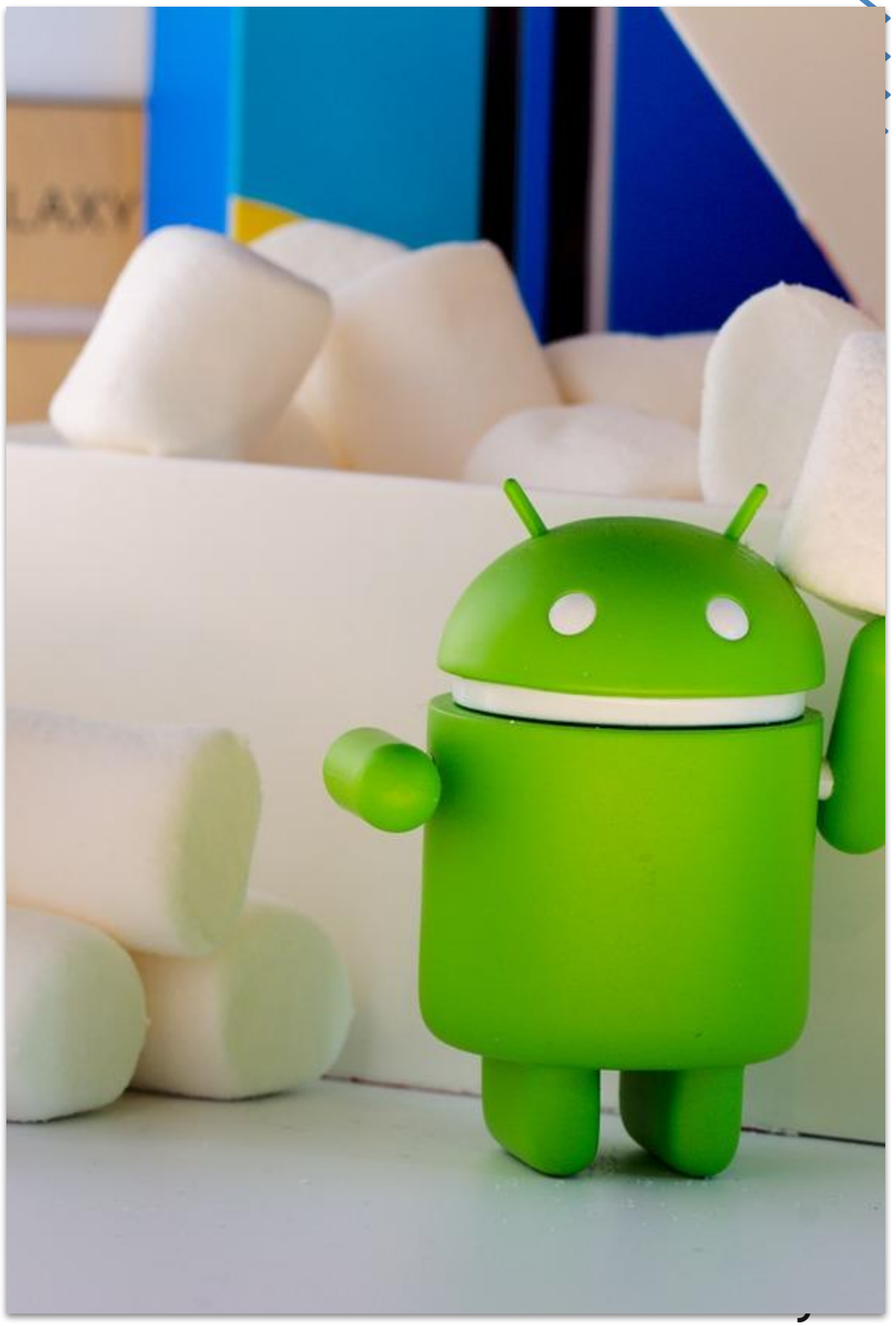
# Los principales elementos de navegación son:

Actividades (Activities)

En una aplicación, las actividades representan las pantallas individuales, siendo cada sección principal implementada como una actividad. Los usuarios pueden cambiar de una actividad a otra a través de elementos de navegación.

# Los principales elementos de navegación son:

Fragmentos (Fragments)
Los fragmentos son módulos de interfaz de usuario que se utilizan para hacer las interfaces más dinámicas en una actividad, permitiendo su adición, eliminación o reemplazo en tiempo de ejecución, lo que da lugar a una navegación adaptable y fluida.
Puedes utilizar FragmentTransaction para agregar, reemplazar o quitar fragmentos en una actividad. Esto es útil para la navegación dentro de una sola actividad.



# Los principales elementos de navegación son:

Navigation Component:
El componente de navegación es una biblioteca de Android Jetpack que proporciona una forma más fácil y declarativa de gestionar la navegación en tu aplicación. Puedes definir rutas de navegación y acciones en un archivo XML y luego utilizar el NavController para navegar entre destinos.
Es una solución que se está adoptado mucho actualmente por su facilidad al diseñar flujos en las aplicaciones.

# Los principales elementos de navegación son:

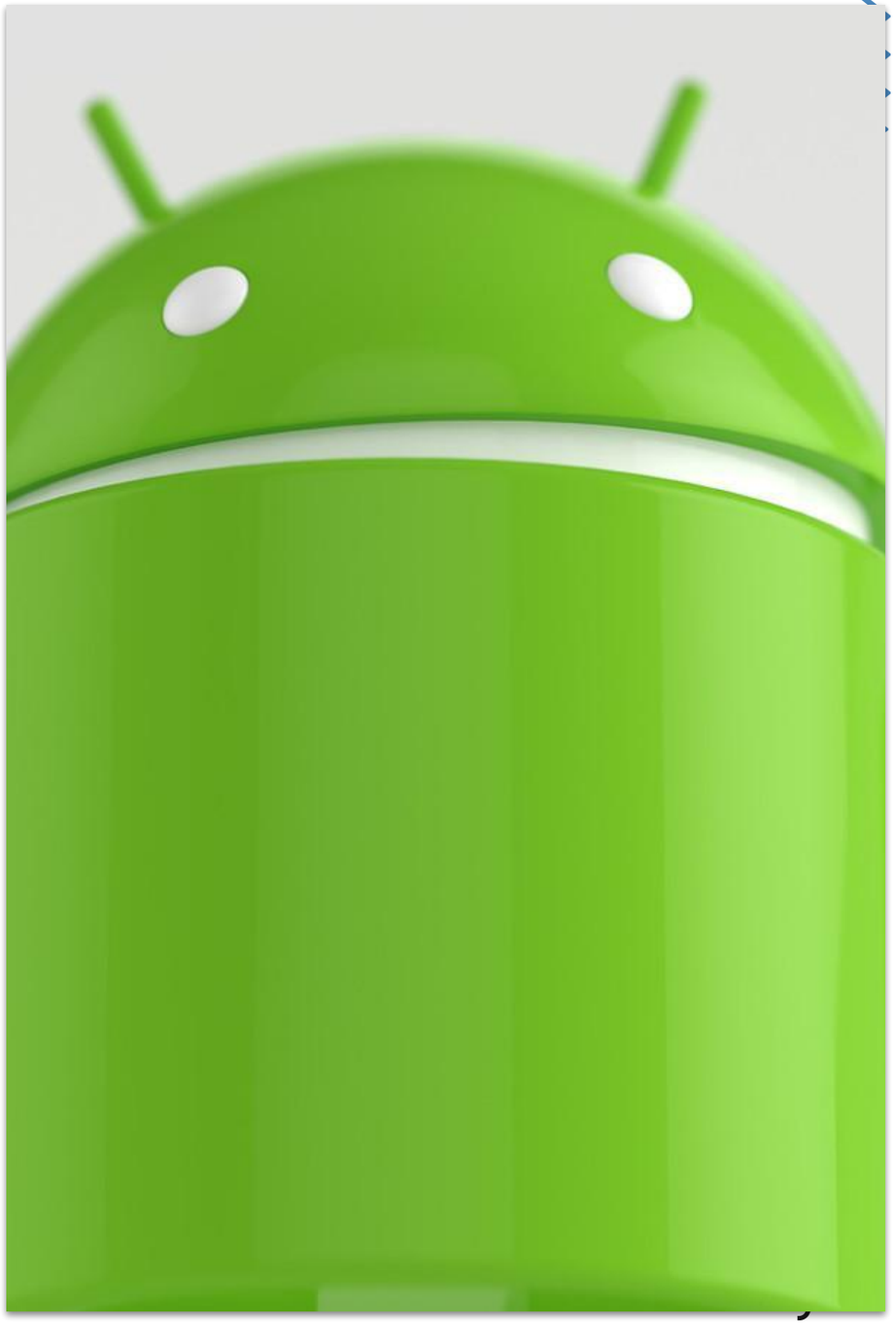
Barra de acción (Action Bar)
La barra de acción es una barra en la parte superior de la pantalla con botones de navegación y funciones en una aplicación.

# Los principales elementos de navegación son:

Pestañas (Tabs)
Las pestañas organizan el contenido de una aplicación en secciones separadas, permitiendo a los usuarios cambiar entre ellas para acceder a distintas partes de la aplicación.

# Los principales elementos de navegación son:

Botones de navegación (Navigation Buttons)
Los botones de navegación en dispositivos Android sin botones físicos consisten en el botón de retroceso, el botón de inicio y el botón de recientes, situados en la parte inferior de la pantalla. Facilitan la navegación entre aplicaciones y pantallas.



# Intent explícitos

# Intent explícitos

Introducción:

Un Intent explícito en Android es un objeto que inicia una actividad específica en una aplicación cuando se conoce previamente el nombre de la actividad a la que se quiere acceder.

Para crear un Intent explícito, se requiere el contexto de la actividad actual y la clase de la actividad de destino. Además, se pueden enviar datos adicionales, como parámetros o valores, a la actividad de destino a través de los extras del Intent.

# Intent explícitos

Veamos cómo lo podemos usar en android:

```
// Crear un Intent explícito
Intent intent = new Intent(ContextoCreado.this, ActividadFinal.class);

// Pasar datos adicionales mediante extras
intent.putExtra("clave", valor);

// Iniciar la actividad de destino
startActivity(intent);
```

# Intent explícitos

En el ejemplo, "ContextoCreado" es el contexto de la actividad actual, "ActividadFina" es la clase de la actividad a la que se desea acceder, "clave" es el nombre del dato extra que se quiere enviar, y "valor" es el valor asociado a esa clave.

# Intent explícitos

Llamar a startActivity(intent) inicia la actividad de destino utilizando el Intent explícito que se ha creado.

# Intent explícitos

Conclusión:

Los Intents explícitos son útiles cuando se sabe de antemano a qué actividad se quiere acceder, como al cambiar entre pantallas dentro de una aplicación.

# LIVE CODING

**Creamos una navegación entre actividades:**

*Creamos 3 actividades A, B y C.  Siendo A la principal/primera actividad. Luego agregamos un indicador con la letra correspondiente a las vistas de cada actividad junto con un botón. Hacer que el botón de A un intent que llame a B y uno en B que llame a C. Correr la aplicacion y ver qué sucede cuando presionamos Atrás en C y qué ocurre si agregamos la instrucción finish() en B luego de terminar la llamada*

**Tiempo: 30 Minutos**

# < > Rompehielo🥶

**Contexto:** 🙌

Hablamos de que existen varias formas de navegar dentro de Android, pero los Fragmentos tienen una muy particular forma de usarse para este tipo de operaciones

**Consigna:** ✍️

Sabiendo como funcionan los fragmentos de las lecciones anteriores: ¿Cómo creen que es la navegaciones entre fragmentos de la misma actividad?

# Fragmentos

# Fragmentos

En el manual anterior, se explicó qué son los fragmentos como partes de la interfaz de usuario. Ahora, se explorará cómo se pueden complementar, crear y navegar.
Los fragmentos son componentes reutilizables que se utilizan para crear interfaces de usuario flexibles y dinámicas en Android. Puedes utilizar FragmentTransaction para agregar, reemplazar o quitar fragmentos en una actividad. Esto es útil para la navegación dentro de una sola actividad.

# Fragment Manager

FragmentManager es una clase en Android que se encarga de gestionar los fragmentos y sus operaciones dentro de una actividad, como agregar, reemplazar o eliminar fragmentos.

*Para obtener la clase FragmentManager en una actividad, se utiliza el método getFragmentManager() o getSupportFragmentManager(), dependiendo de la versión de Android y del tipo de soporte que se necesite.*

alkemy

# Fragment Manager

Se pueden llevar a cabo varias operaciones con fragmentos. Solemos llamar las transacciones o Transactions, y se usan para realizar operaciones como agregar, reemplazar y eliminar fragmentos. A continuación, se presentan ejemplos de código para estas operaciones.

# Agregar un fragmento

Para agregar un fragmento a una actividad, se utiliza el método beginTransaction() del FragmentManager para iniciar una transacción. Luego, se utiliza el método add() para especificar el contenedor y el fragmento que se añadirá.

*Antes de usarlo, es necesario definir el fragmento que se desea agregar. Esto implica crear una clase que extienda Fragment y personalizar su contenido y comportamiento. Por ejemplo, podemos crear un fragmento llamado "MiPrimerFragmento".*

# Agregar un fragmento

```
public class MiPrimerFragmento extends Fragment {
   // Constructor por defecto
   public MiPrimerFragmento() {
      // Required empty public constructor
   }

   @Override
   public View onCreateView(LayoutInflater inflater, ViewGroup container,
                            Bundle savedInstanceState) {
      // Inflar el layout del fragmento
            return  inflater.inflate(R.layout.fragment_mi_primer_fragmento,
container, false);
   }
}
```

# Agregar un fragmento

En la actividad donde se desea agregar un fragmento, se debe utilizar el FragmentManager para iniciar una transacción y agregar el fragmento al contenedor deseado. Esto se puede hacer en el método onCreate() de la actividad.

```
public class MiActividad extends AppCompatActivity {

@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_mi_actividad);
// Obtener el FragmentManager
FragmentManager fragmentManager = getSupportFragmentManager();
// Iniciar una transacción
FragmentTransaction fragmentTransaction = fragmentManager.beginTransaction();
// Crear una instancia del fragmento
MiFragmento fragmento = new MiPrimerFragmento();
// Agregar el fragmento al contenedor (por ejemplo, un FrameLayout con id
"contenedor_fragmento")
fragmentTransaction.add(R.id.contenedor_fragmento, fragmento);
// Confirmar la transacción
fragmentTransaction.commit();
}
}
```

# Agregar un fragmento

En este ejemplo, se muestra cómo usar el FragmentManager para agregar un fragmento en la actividad. Se obtiene el FragmentManager con getSupportFragmentManager(), se inicia una transacción con beginTransaction(), se crea una instancia del fragmento, se utiliza add() para especificar el contenedor y el fragmento, y se confirma la transacción con commit(). Asegúrate de reemplazar "R.layout.fragment_mi_primer_fragmento" con el nombre correcto del layout de tu fragmento y de tener el contenedor adecuado en tu actividad.

# Reemplazar un fragmento

En lugar de agregar un fragmento sobre otro, es posible reemplazar un fragmento existente con uno nuevo utilizando el método replace() del FragmentManager y especificando el contenedor y el nuevo fragmento.

Para utilizarlo, primero asegúrate de tener definidos los fragmentos que deseas utilizar, al igual que en el ejemplo anterior.

# Reemplazar un fragmento

En la actividad donde se desea reemplazar un fragmento, se utiliza el FragmentManager para iniciar una transacción y sustituir el fragmento existente por uno nuevo. Esto se realiza en el método onCreate() de la actividad.

# Reemplazar un fragmento

```
public class MiActividad extends AppCompatActivity {

@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_mi_actividad);
// Obtener el FragmentManager
FragmentManager fragmentManager = getSupportFragmentManager();
// Iniciar una transacción
FragmentTransaction fragmentTransaction = fragmentManager.beginTransaction();
// Crear una instancia del nuevo fragmento
UnNuevoFragmento unNuevoFragmento = new UnNuevoFragmento();
// Reemplazar el fragmento existente por el nuevo fragmento en el contenedor (por ejemplo, un
FrameLayout con id "contenedor_fragmento")
fragmentTransaction.replace(R.id.contenedor_fragmento, unNuevoFragmento);
// Confirmar la transacción
fragmentTransaction.commit();
}
}
```

# Reemplazar un fragmento

En el ejemplo proporcionado, después de obtener el FragmentManager y comenzar una nueva transacción, se crea una instancia del nuevo fragmento a reemplazar. Luego, se utiliza el método replace() de la transacción para especificar el contenedor (como un FrameLayout con el id "contenedor_fragmento") y el nuevo fragmento, lo que sustituirá el fragmento existente en el contenedor. Para finalizar, se confirma la transacción mediante commit(). Asegúrate de sustituir "R.id.contenedor_fragmento" con el id del contenedor adecuado en tu actividad y de personalizar el código según las necesidades de tu aplicación.

# Eliminar un fragmento

Puedes eliminar un fragmento específico de una actividad utilizando el método remove(). Para hacerlo, debes dirigirte a la actividad en cuestión, utilizar el FragmentManager para iniciar una transacción y eliminar el fragmento deseado.

En este ejemplo eliminaremos el primer ejemplo que usamos (MiPrimerFragmento):

# Eliminar un fragmento

```
public class MiActividad extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_mi_actividad);
        // Obtener el FragmentManager
        FragmentManager fragmentManager = getSupportFragmentManager();
        // Iniciar una transacción
        FragmentTransaction fragmentTransaction = fragmentManager.beginTransaction();
        // Obtener el fragmento existente que se desea eliminar
        Fragment MiPrimerFragmento = fragmentManager.findFragmentById(R.id.contenedor_fragmento);
        if (MiPrimerFragmento != null) {
            // Eliminar el fragmento existente
            fragmentTransaction.remove(miPrimerFragmento);
        }
    // Confirmar la transacción
        fragmentTransaction.commit();
    }
}
```

# Eliminar un fragmento

En este ejemplo, se utiliza el FragmentManager para iniciar una transacción. Luego, se emplea el método findFragmentById() para obtener el fragmento existente que se desea eliminar. Si el fragmento no es nulo, se utiliza el método remove() dentro de la transacción, pasando el fragmento como parámetro. Finalmente, se confirma la transacción mediante commit().

# Realizar transacciones de fragmentos

Cuando trabajamos con múltiples fragmentos, podemos agruparlos en una transacción de fragmentos que permite realizar operaciones como agregar, reemplazar o eliminar fragmentos. Estas transacciones se inician utilizando beginTransaction() del FragmentManager y se confirman con commit().

*Para implementarlo, asegúrate de identificar los fragmentos que deseas agrupar. Luego, en tu archivo XML de diseño, agrega un FragmentContainerView como contenedor para tus fragmentos.*

alkemy

# Realizar transacciones de fragmentos

Por ejemplo:

```
<androidx.fragment.app.FragmentContainerView
                                                    android:id="@+id/contenedor_fragmentos"
                          android:name="androidx.navigation.fragment.NavHostFragment"
                                                        android:layout_width="match_parent"
                                                       android:layout_height="match_parent"
                                                                   app:defaultNavHost="true"
    app:navGraph="@navigation/nav_graph" />
```

# Realizar transacciones de fragmentos

En el ejemplo, se añadió un FragmentContainerView con el ID "contenedor_fragmentos". Se configuraron atributos como android:name para identificarlo como un NavHostFragment, app:defaultNavHost="true" para que sea el host de navegación predeterminado, y app:navGraph="@navigation/nav_graph" para especificar el gráfico de navegación a utilizar.

*En la actividad, se utiliza el FragmentManager para realizar transacciones y agregar fragmentos en el FragmentContainerView.*

alkemy

# Realizar transacciones de fragmentos

```
public class MiActividad extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_mi_actividad);
        // Obtener el FragmentManager
        FragmentManager fragmentManager = getSupportFragmentManager()
        // Iniciar una transacción
        FragmentTransaction fragmentTransaction = fragmentManager.beginTransaction()
        // Crear instancias de los fragmentos que deseas agregar
        Fragment fragmento1 = new Fragmento1();
        Fragment fragmento2 = new Fragmento2();
        // Agregar los fragmentos al contenedor (FragmentContainerView)
        fragmentTransaction.add(R.id.contenedor_fragmentos, fragmento1);
        fragmentTransaction.add(R.id.contenedor_fragmentos, fragmento2);

        // Confirmar la transacción
        fragmentTransaction.commit();
    }
}
```

# Realizar transacciones de fragmentos

En este ejemplo, se obtuvo el FragmentManager, se inició una transacción y se crearon instancias de los fragmentos Fragmento1 y Fragmento2. Luego, se agregaron al contenedor FragmentContainerView utilizando el método add() de la transacción, y finalmente se confirmó la transacción mediante commit().

Asegúrate de reemplazar "R.id.contenedor_fragmentos" con el ID correcto del FragmentContainerView en tu actividad y ajustar el código según las necesidades y requisitos específicos de tu aplicación.

# LIVE CODING

**Crear transacciones:**

*Vamos a crear una vista que contiene un fragmento y dos fragmentos que vamos a usar para transicionar. El primer fragmento va a tener un botón que nos mueve al segundo fragmento. Luego crear las transacciones para eliminar el primer fragmento. El segundo fragmento tiene un botón que hace que se reemplace con el primero nuevamente.*

**Tiempo: 30 Minutos**

# Navigation Component

# Navigation Component

Introducción:

El Navigation Component es un componente fundamental de la biblioteca de navegación en Android. Su propósito es simplificar la implementación de la navegación en una aplicación al proporcionar una estructura coherente y sencilla para definir destinos de navegación, gestionar el historial de navegación, realizar transiciones entre destinos y transmitir argumentos de navegación de forma segura.

# Navigation Component

Antes de mostrar un ejemplo en el código, asegúrate de haber añadido las dependencias necesarias en tu archivo build.gradle para utilizar Navigation Component. Por ejemplo:

```
dependencies {
    def nav_version = "2.4.0"

    // Agregar las dependencias de Navigation Component
    implementation "androidx.navigation:navigation-fragment-ktx:$nav_version"
    implementation "androidx.navigation:navigation-ui-ktx:$nav_version"
}
```

# Navigation Component

Ahora, debes crear un archivo XML llamado "nav_graph.xml" en el directorio res/navigation de tu proyecto para definir el gráfico de navegación. En este archivo, puedes especificar los fragmentos, actividades y acciones de navegación que deseas incluir en tu aplicación. Por ejemplo:

En este ejemplo, se definen dos fragmentos, HomeFragment y DetailsFragment, junto con una acción de navegación llamada "action_home_to_details" que permite la transición desde el fragmento de inicio (HomeFragment) al fragmento de detalles (DetailsFragment).

```xml
<?xml version="1.0" encoding="utf-8"?>
<navigation
xmlns:android="http://schemas.android.com/apk/res/andro
id"
    xmlns:app="http://schemas.android.com/apk/res-auto"
        xmlns:tools="http://schemas.android.com/tools"
            app:startDestination="@id/homeFragment">

                <fragment
                    android:id="@+id/homeFragment"
            android:name="com.example.app.HomeFragment"
                    android:label="Home"
            tools:layout="@layout/fragment_home" />

                <fragment
                    android:id="@+id/detailsFragment"
        android:name="com.example.app.DetailsFragment"
                    android:label="Details"
            tools:layout="@layout/fragment_details" />

                <action
            android:id="@+id/action_home_to_details"
            app:destination="@id/detailsFragment" />

</navigation>
```

# Navigation Component

En la actividad principal, debes configurar el Navigation Component y vincularlo al contenedor de fragmentos. Esto se realiza, por ejemplo, en el método onCreate() de tu actividad.
En este ejemplo, se obtiene el NavController utilizando Navigation.findNavController(this, R.id.nav_host_fragment), donde "nav_host_fragment" es el ID del contenedor de fragmentos en el diseño de la actividad.
También se configura el Navigation Component con la ActionBar utilizando NavigationUI.setupActionBarWithNavController(this, navController). Esto asegura que el botón de navegación en la ActionBar funcione correctamente y permita la navegación hacia atrás en el gráfico de navegación.

```
public class MainActivity extends AppCompatActivity {

    private NavController navController;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        // Obtén el NavController desde el Navigation Component
        navController = Navigation.findNavController(this, R.id.nav_host_fragment);
        // Configura el Navigation Component con la ActionBar
        NavigationUI.setupActionBarWithNavController(this, navController);
    }

    @Override
    public boolean onSupportNavigateUp() {
        // Navega hacia atrás cuando se presiona el botón de navegación de la ActionBar
        return navController.navigateUp() || super.onSupportNavigateUp();
    }
}
```

# Navigation Component

En tu diseño de actividad (activity_main.xml), asegúrate de incluir un contenedor de fragmentos con el ID correspondiente, como se muestra en el ejemplo.

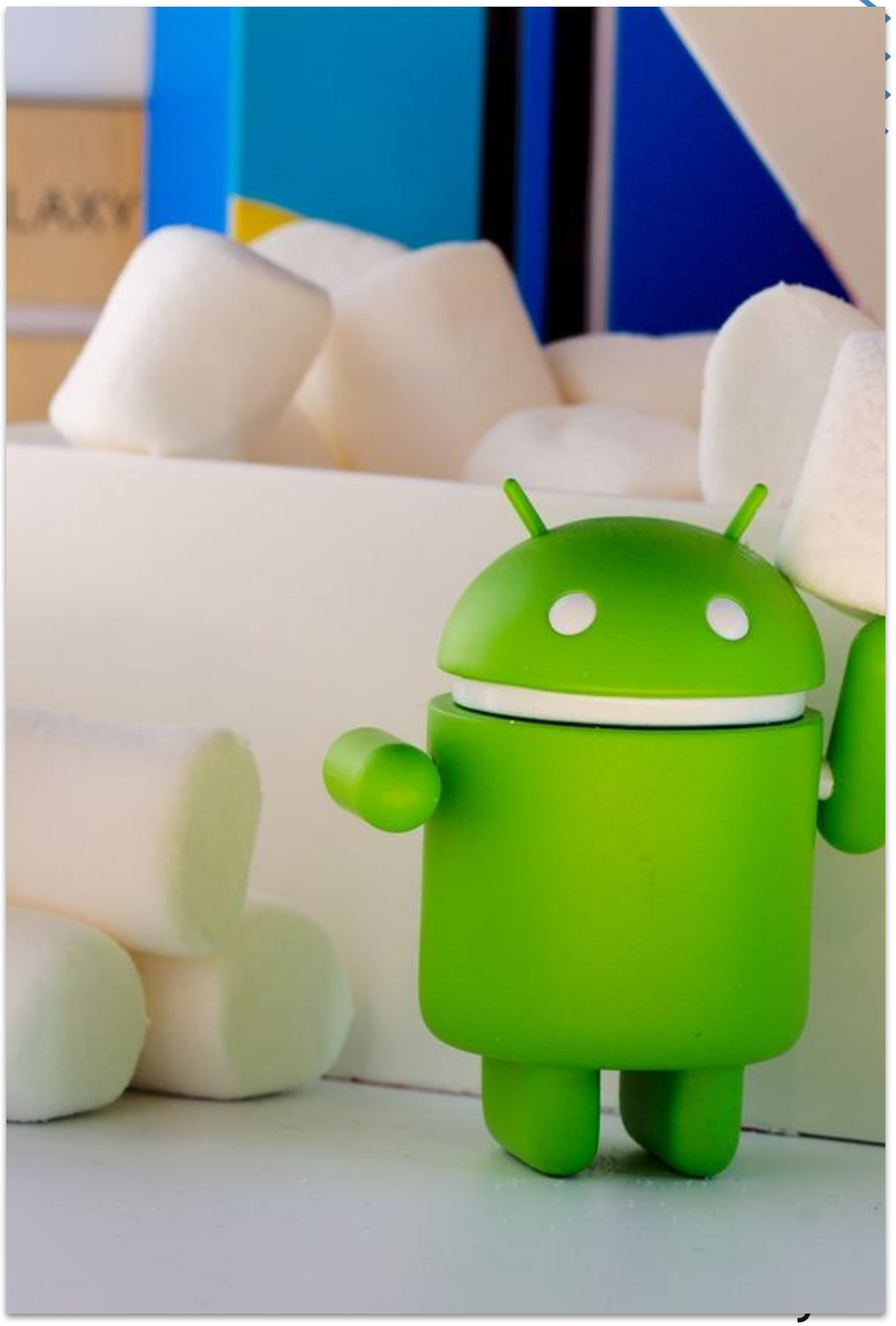
```
<androidx.fragment.app.FragmentContainerView
                                        android:id="@+id/nav_host_fragment"
              android:name="androidx.navigation.fragment.NavHostFragment"
                                        android:layout_width="match_parent"
                                       android:layout_height="match_parent"
                                                  app:defaultNavHost="true"
    app:navGraph="@navigation/nav_graph" />
```

En el ejemplo, se emplea un FragmentContainerView como contenedor de fragmentos con el ID "nav_host_fragment". Además, se configura con app:defaultNavHost="true" y app:navGraph="@navigation/nav_graph" para indicar que este contenedor actúa como el host de navegación predeterminado y utiliza el gráfico de navegación definido en el archivo "nav_graph.xml".

# Interacciones de usuario

# Interacciones de usuario

En el Navigation Component, las interacciones de usuario se gestionan principalmente a través de acciones y controladores de navegación. A continuación, se presentan algunas formas habituales de gestionar las interacciones de usuario utilizando Navigation Component:



# Interacciones de usuario

Navegación mediante acciones

Puedes definir acciones de navegación en tu gráfico de navegación (nav_graph.xml) y luego invocar esas acciones en respuesta a las interacciones de usuario. Por ejemplo, al tener un botón en un fragmento que debe llevar al usuario a otro fragmento, puedes crear una acción de navegación en el gráfico de navegación y utilizar Navigation.findNavController(view).navigate(R.id.action_id) para navegar a través de esa acción cuando se hace clic en el botón.

# Interacciones de usuario

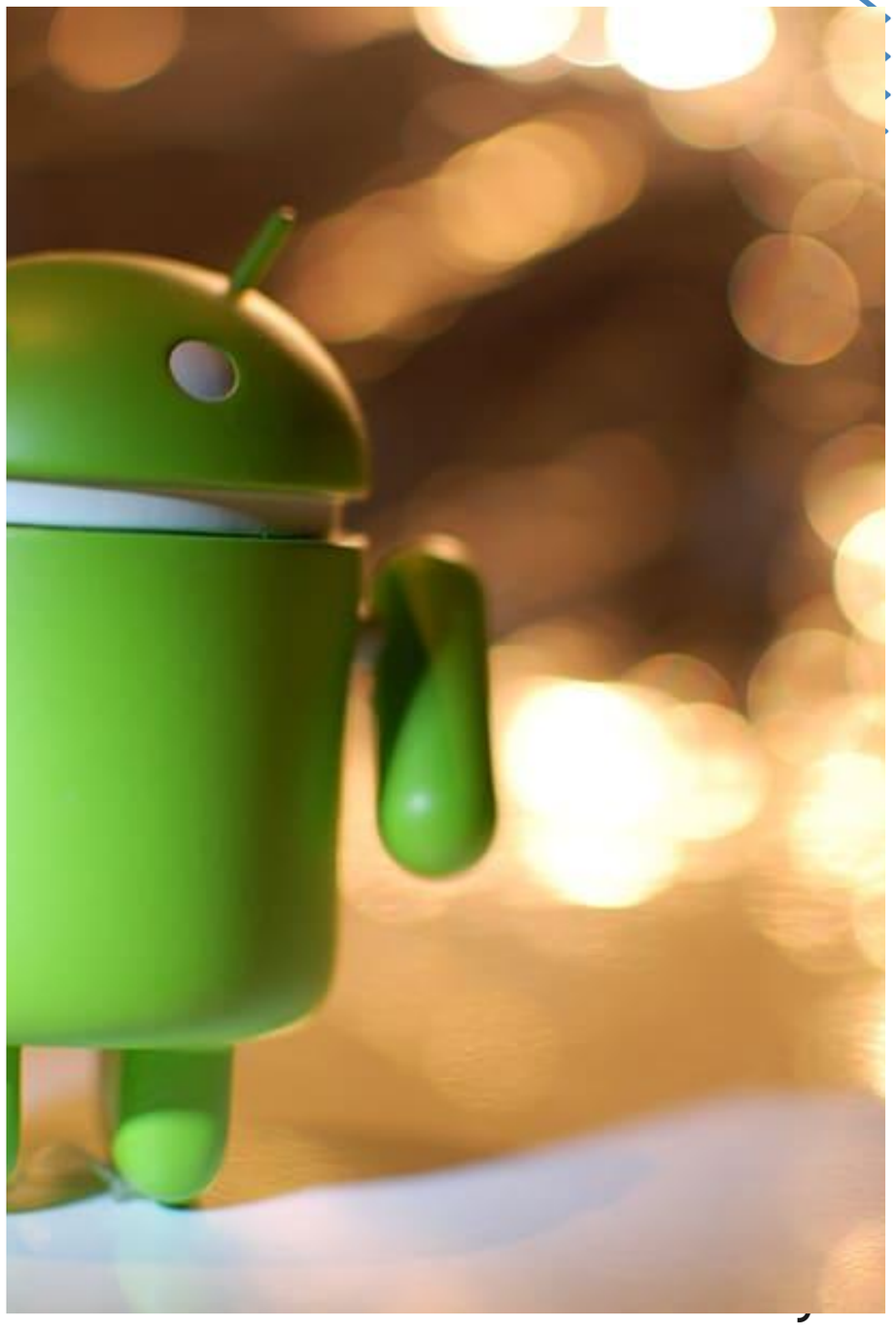
Controladores de navegación

El NavController te permite gestionar las interacciones de usuario de forma programática. Puedes utilizar navController.navigate(R.id.destination_id) para navegar a un fragmento o actividad específica en respuesta a una interacción de usuario. Además, métodos como navigateUp() o popBackStack() te permiten controlar el flujo de navegación hacia atrás.

# Interacciones de usuario

Argumentos de navegación

Para pasar datos entre fragmentos o actividades durante la navegación, puedes utilizar argumentos de navegación. Definir argumentos en el gráfico de navegación y establecerlos antes de la navegación a través de actionDestination().setArguments(args) o navController.getCurrentDestination().getArguments().putXXX(key, value) te permite transferir datos importantes de una pantalla a otra.



# Interacciones de usuario

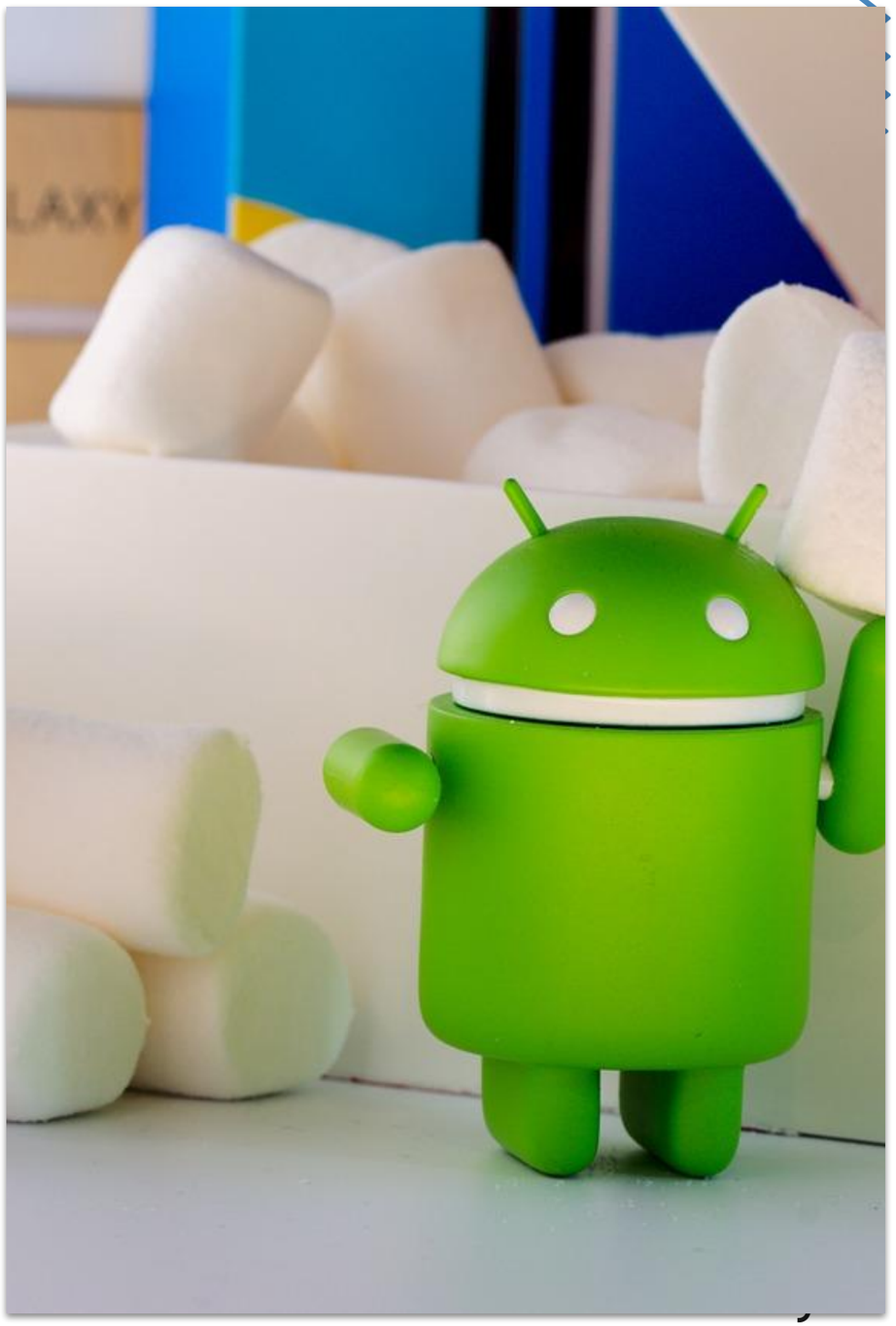
Callbacks de navegación

Puedes utilizar callbacks de navegación, como OnDestinationChangedListener, para ejecutar acciones personalizadas antes o después de la navegación, como actualizar la interfaz de usuario, cambiar el título de la ActionBar y más. Además, Navigation Component ofrece características avanzadas como animaciones de transición, navegación profunda, navegación por argumentos y otras funcionalidades para gestionar interacciones de usuario de manera más sofisticada.

# ViewBinding y NullPointerException

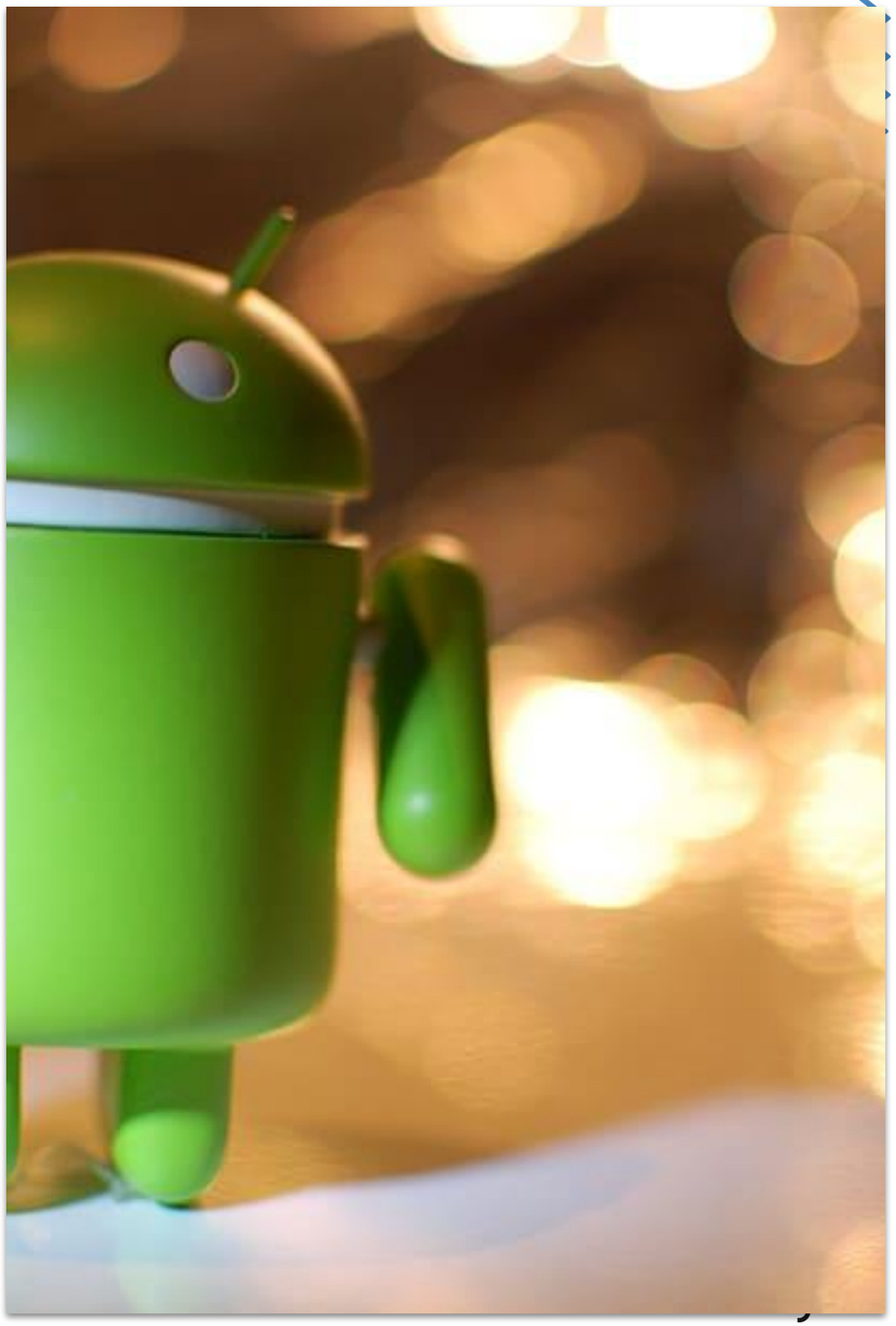
# ViewBinding

View Binding es una característica de Android que simplifica la vinculación de vistas en tus diseños XML con el código de tu aplicación. Proporciona una forma segura y eficiente de acceder a las vistas de tu diseño sin la necesidad de utilizar métodos como findViewById.
Vamos a explorar mucho más como implementarlo en próximos módulos.



# NullPointerException

Este es un error muy común cuando trabajamos con vistas, antes era común que ocurrieran cuando intentabas acceder a vistas en tu diseño (por ejemplo, botones, texto, etc.) utilizando métodos como findViewById. Si la vista con el ID especificado no existía en el diseño o tenía un ID diferente, se generaba una NullPointerException cuando intentabas acceder a esa vista.



# Relación entre los dos

La relación es que View Binding puede ayudar a prevenir NullPointerExceptions relacionados con la falta de referencias de vistas nulas en tus actividades y fragmentos. View Binding aborda este problema generando automáticamente clases de enlace de vista para cada archivo de diseño XML en tu proyecto. Estas clases de enlace de vista contienen referencias directas a las vistas dentro de ese diseño. Dado que estas referencias se generan automáticamente, es mucho menos probable que obtengas NullPointerExceptions relacionadas con las vistas en tus actividades o fragmentos.

Ejercicio N° 1

# Buscar en Google desde un formulario

# Buscar en Google desde un formulario

**Consigna:** ✍️

Permitir al usuario escribir un término en un EditText, presionar un botón, y abrir otra actividad que muestre los resultados en un WebView.

**Enfoque:** Intents + WebView + XML sencillo

**Tiempo** 🕐**:** *20/30 minutos*

**Estructura de archivos:** ⚙️

MainActivity.java → contiene el campo de búsqueda

WebViewActivity.java → muestra los resultados

activity_main.xml → interfaz con EditText y botón

activity_web_view.xml → interfaz con WebView

# ¿Alguna consulta?

alkemy

# Resumen

*¿Qué logramos en esta clase?*

- ✓ Navigation Components
- ✓ interacción de usuario
- ✓ ViewBingin y NullPointerException
- ✓ Relación entre los dos
- ✓ Métodos de navegacion
- ✓ Elementos en navegación
- ✓ Intents explicitos
- ✓ Fragmentos
- ✓ Transacciones
- ✓ Navegacion de Fragmentos

# ¡Ponte a prueba!

Momento de ejercitación

*Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las* ***actividades disponibles en la plataforma.*** *Estas propuestas son clave para afianzar lo trabajado y* ***forman parte obligatoria del recorrido de aprendizaje.***

💪 Análisis de caso ——— 💪 Selección Múltiple

💪 Comprensión lectora

*Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.*

alkemy

# < ¡Muchas gracias! >

alkemy